SOC L1 LAB · INC-004

Attack Setup Guide: INC-004

Two root persistence paths, and the legitimate work that looks the same

INCIDENT	INC-004, Cron and systemd persistence
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise, sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	Critical

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Authorization and safety
2. Topology and assumptions
3. Phase 0: Clean state
4. Phase 1: Legitimate baseline (required)
5. Phase 2: Cron persistence
6. Phase 3: systemd persistence
7. Phase 4: Enable and execute
8. Phase 5: Evidence capture
9. Phase 6: Cleanup
10. Expected detections
11. Variations
12. Why this is realistic
13. Legal and ethical notes

A step-by-step script for reproducing INC-004 inside the lab. It is for testing detection and response, not for use against systems you do not own.

---

1. Authorization and safety

Rule	Detail
Scope	Only the `soc-victim` VM on `10.0.2.0/24`
Never	Touch any other host or network
Never	Use real credentials, malware, or employer data
Authorization	You own the host and the VM
Reversibility	Snapshot, then remove every file and disable every timer
Payload	`/usr/bin/id` writing to `/tmp`. No external C2

The scenario models an insider or a compromised administrator session. It is deliberately an authorized admin action, because that is what makes change control the only reliable discriminator.

---

2. Topology and assumptions

```
Host 10.0.2.2
  ├── Wazuh manager / indexer / dashboard
  ├── analyst SSH session (sudo)
  └── QEMU user network
        └── soc-victim 10.0.2.15
              cron, systemd
              FIM: /etc (realtime)
              auditd: execve + file watches
              rules: 100129, 100130
```

Assumptions:

- Base lab from INC-001 is running.
- `cron` is installed and active.
- `/etc` is in FIM realtime.
- Rules 100129/100130 are deployed.
- `analyst` has passwordless sudo.
- No `updates` unit or `/etc/cron.d/updates` exists.

```
export LAB="$HOME/Documents/Eterna/soc-l1-lab"
export VMKEY="$LAB/lab/vm/keys/id_ed25519"
```

```
export VM="analyst@127.0.0.1"
export VMPORT=2222
```

---

3. Phase 0: Clean state

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
systemctl is-active cron
ls -1 /etc/cron.d
systemctl list-timers --all | grep -i updates || echo "no updates timer"'
```

Expected: cron active. No `updates` files.

Snapshot:

```
cd "$LAB/lab/vm"
./snapshot.sh save pre-inc004
```

---

4. Phase 1: Legitimate baseline (required)

Run this before the attack so the false positives are known.

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
echo "0 3 * * * root /usr/bin/true" | sudo tee /etc/cron.d/apt-maintenance >/dev/null
sleep 4
echo "0 4 * * * root /usr/bin/true" | sudo tee /etc/cron.d/apt-maintenance >/dev/null
sleep 4
sudo systemctl disable cron >/dev/null
sleep 3
sudo systemctl enable cron >/dev/null'
```

Expected:

- 100129 twice: the legitimate cron file added and modified.
- 100130 once: the legitimate `cron.service` enable symlink.

Remove the test cron file:

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" 'sudo rm -f /etc/cron.d/apt-maintenance'
```

Record the counts. These are not failures. They are the change-monitoring baseline.

---

5. Phase 2: Cron persistence

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
echo "* * * * * root /usr/bin/id > /tmp/cron-proof 2>&1" \
  | sudo tee /etc/cron.d/updates
sudo sha256sum /etc/cron.d/updates'
```

Expected hash:

```
991523eaf5c220a6217557fafc3be04d13f5e496c85f1944641efd71fca05651
```

Expected detections:

- 100129 on `/etc/cron.d/updates`.
- 80792 `tee /etc/cron.d/updates`.
- 5402 the sudo command.

The job runs at the next minute boundary.

---

6. Phase 3: systemd persistence

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo tee /etc/systemd/system/updates.service >/dev/null <<"EOF"
[Unit]
Description=Updates

[Service]
Type=oneshot
ExecStart=/bin/sh -c "id > /tmp/systemd-proof 2>&1"
EOF

sudo tee /etc/systemd/system/updates.timer >/dev/null <<"EOF"
[Unit]
Description=Updates timer

[Timer]
OnBootSec=30s
OnUnitActiveSec=60s
Unit=updates.service

[Install]
WantedBy=timers.target
EOF

sudo sha256sum /etc/systemd/system/updates.service \
  /etc/systemd/system/updates.timer'
```

Expected hashes:

```
1674fae2cbf55c8e5cbb3bb8889409b951811e5a1c26c3e8f35c9efc6760c574  updates.service
b58a06d878ddb229f8bfed5eaebf1d32341ee345d0897b765868f7d16c6f0129  updates.timer
```

Expected detections:

- 100130 on `updates.service`.
- 100130 on `updates.timer`.
- 80792 `tee` for both.

---

7. Phase 4: Enable and execute

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo systemctl daemon-reload
sudo systemctl enable --now updates.timer
systemctl is-active updates.timer'
```

Expected:

- 100130 on `timers.target.wants/updates.timer`.
- 80792 `systemctl daemon-reload` and `systemctl enable --now updates.timer`.
- timer active.

Wait at least 75 seconds:

```
sleep 75
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
echo "cron-proof:"; sudo cat /tmp/cron-proof 2>/dev/null || echo missing
echo "systemd-proof:"; sudo cat /tmp/systemd-proof 2>/dev/null || echo missing'
```

Expected:

```
cron-proof:
uid=0(root) gid=0(root) groups=0(root)
systemd-proof:
uid=0(root) gid=0(root) groups=0(root)
```

The proof is the difference between a suspicious file and an executed root payload.

---

8. Phase 5: Evidence capture

```
date -u +%FT%TZ > /tmp/inc004_end

docker exec single-node-wazuh.manager-1 cat /var/ossec/logs/alerts/alerts.json \
  > /tmp/alerts_all.json

python3 - <<'PY'
import json, datetime
start=datetime.datetime.fromisoformat("2026-10-31T23:17:20+00:00")
end=datetime.datetime.fromisoformat("2026-10-31T23:19:30+00:00")
rows=[]
for line in open('/tmp/alerts_all.json'):
    try: a=json.loads(line)
    except: continue
    if a.get('agent',{}).get('name')=='soc-victim' and \
       start <= datetime.datetime.fromisoformat(a['timestamp']) <= end:
        rows.append(a)
json.dump(rows, open('docs/evidence/INC-004-alerts.json','w'), indent=1)
print(len(rows), "alerts")
PY

ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo sha256sum /etc/cron.d/updates \
  /etc/systemd/system/updates.service \
  /etc/systemd/system/updates.timer
sudo cat /tmp/cron-proof /tmp/systemd-proof' \
  | tee docs/evidence/INC-004-hashes-and-proof.txt
```

---

9. Phase 6: Cleanup

```
ssh -i "$VMKEY" -p "$VMPORT" "$VM" '
sudo systemctl disable --now updates.timer
sudo rm -f /etc/systemd/system/updates.service /etc/systemd/system/updates.timer
sudo systemctl daemon-reload
sudo rm -f /etc/cron.d/updates
sudo rm -f /tmp/cron-proof /tmp/systemd-proof
systemctl list-timers --all | grep -i updates || echo "no updates timer"
ls /etc/cron.d'
```

Expected: no updates timer, no `updates` cron file. cron remains installed and active.

If you snapshotted:

```
cd "$LAB/lab/vm"
./snapshot.sh load pre-inc004
```

10. Expected detections

Phase	Rule	Expected
Legit cron file	100129	added and modified
Legit `enable cron`	100130	enable symlink
Cron payload	100129	`/etc/cron.d/updates`
Service file	100130	`updates.service`
Timer file	100130	`updates.timer`
Enable timer	100130	`timers.target.wants/updates.timer`
All writes	80792	`tee` with paths
Enable	80792	`systemctl` command
Actor	5402	sudo command
Execution	proof	`uid=0(root)`

11. Variations

11.1 User crontab

```
crontab -e    # or: (crontab -l; echo "* * * * * /usr/bin/id") | crontab -
```

This writes to `/var/spool/cron/crontabs/<user>`, which is not covered by 100129. Add FIM/audit coverage to detect it. This is the most important gap to test.

11.2 Service without a timer

A `.service` with `WantedBy=multi-user.target` runs at boot. It produces the same 100130 file event. The difference is the enable target and the execution trigger.

11.3 Direct symlink creation

An attacker can create `/etc/systemd/system/multi-user.target.wants/foo.service` directly instead of running `systemctl enable`. FIM still catches the symlink, but the audit trail lacks the `systemctl` command. Another reason to treat the file event as the primary signal and the command as context.

11.4 Deletion / anti-forensics

```
sudo rm /etc/cron.d/updates
```

The deletion is 553, which 100129/100130 do not match. Add a deletion rule if you want visibility into cleanup.

11.5 Package-installed cron

Installing a package can add `/etc/cron.d/<name>` (the image already has `php` and `e2scrub_all`). This is a legitimate 100129 hit. Match it to the package transaction.

---

12. Why this is realistic

- **Two mechanisms, one actor**
  Real persistence often uses more than one path.
- **Root execution is trivial**
  Cron and systemd both run as root by design.
- **The file writes look like administration**
  `tee`, `systemctl`, `daemon-reload`.
- **The rules cannot tell intent**
  A legitimate cron job and a malicious one produce the same alert.
- **Cleanup is invisible**
  Deletion is not covered by the rules.

---

13. Legal and ethical notes

- Run this only on the VM you created.
- The payload is `id` to a local file. No real C2 or malware.
- Do not publish real unit names or hashes from production.
- For a real engagement, use written authorization and a scope document.